

NAME

bdd – Bit stream copy and modify

SYNOPSIS

bdd [*STREAM_PATTERN*] [*INPUT_PATTERN*] [*OUTPUT_PATTERN*] [*OPTIONS*]

DESCRIPTION

bdd (binary-dd) inspects, transforms, unpacks, manipulates, interleaves, and packs bitstreams with arbitrary (non-byte-aligned) bit precision.

By default, **bdd** reads bytes from standard input and writes them unchanged to standard output, functioning like an exact 8-bit copy tool. With options and patterns, its behavior can be extended to manipulate inputs and outputs down to single-bit accuracy.

bdd models input and output as continuous streams of bit units. A bit unit is a repeating sequence containing a constant number of bits. Units can start at arbitrary bit offsets, be spaced by periodic gaps, or reside inside repeating physical containers (raw units).

Each unit can be unpacked into structured, typed fields called a *tuple* (analogous to a row in a relational database). Fields within a tuple may represent unsigned or signed integers, floating-point numbers (from IEEE double-precision down to sub-byte AI float formats), UTF-8 characters, or raw bytes.

Tuples can be transformed in flight through an ordered pipeline of operations (arithmetic, bitwise masks, shifts, rounding, clamping, and conditional filtering) and repacked into output bit units or formatted into human- and machine-readable text formats (hex, bits, decimal integers, CSV, NDJSON, or ANSI visual dumps).

bdd also supports secondary merge streams, allowing data from one or more secondary files to be interleaved round-robin with the primary stream, as well as field demuxing into separate destination files.

The name **bdd** comes from *binary-dd* because it performs bitstream-level operations analogous to what **dd**(1) does for block-aligned data.

OPTIONS**Positional Pattern Arguments:**

bdd supports optional positional arguments for rapid bitstream slicing and tuple unpacking:

STREAM_PATTERN

Stream I/O mapping pattern defining input and output framing (e.g. "8->3", "123:8[2:4]+8-> 5B:8[2:4]", or "[2:4:2] -> 4"). See **UNIFIED STREAM I/O MAPPING SYNTAX** below. If the first positional argument does not match a stream pattern (e.g. 4U4U), it is interpreted as the *INPUT_PATTERN*.

INPUT_PATTERN

Tuple unpack pattern (e.g. "8U", "4U4U", "sync:11u,version:2u"). Equivalent to **--input-pattern**.

OUTPUT_PATTERN

Tuple pack pattern (e.g. "8U", "4U4U", "8U, x, 8U"). Equivalent to **--output-pattern**. Note: when **--input-tuples** is used, input fields are parsed from commas, so a single positional pattern specifies the output packing pattern.

Input and Output File Selection

The default is to use standard input and standard output.

--input-file=FILENAME

Selects input file to use instead of standard input pipe. Cannot be used with **--input-zeros**, **--input-ones**, **--input-random**, or **--input-counter**. **--input-integers** and **--input-tuples** take their input from the input file. Filename can also be "-" to specify standard input pipe (default).

--output-file=FILENAME

Selects output file to use instead of standard output pipe. Filename can also be "-" to specify standard output pipe (default). Output file must not already exist; **bdd** will not overwrite an existing file without explicit redirection.

Input Units, Skips, and Gaps

A unit is a constant number of bits that repeats in the input stream. All stream dimensions are measured strictly in bits:

--input-unit=BITS

Breaks input into active units of BITS. If input is less than an even amount of units, the most significant bits of the last unit are taken from the input file and the least significant bits are filled with zeroes. Input unit defaults to 8 bits, or to the remaining bits in **--input-raw-unit** after **--input-offset**. If **--input-pattern** is defined, the combined bit size of the **--input-pattern** defines the input unit. You must not define both **--input-unit** and **--input-pattern**.

--input-skip-bits=BITS

Skip BITS bits from the beginning of the input before reading the first unit. Supports 64-bit integer values (up to 18.4 Exabits / 2.3 Exabytes), human-friendly size suffixes, and arbitrary multiplication expressions using * or x (e.g. 1000000*24 to skip one million 24-bit units, 1000*1000*24, or 1920x1080*24): binary multiples (K, M, G, T, P, E or Ki, Mi, Gi, Ti, Pi, Ei, e.g. 10G = 10*1024³ bits), decimal multiples (KB, MB, GB, TB, PB, EB), and byte-scaled multiples (e.g. 10GiB = 10*1024³*8 bits, 100B = 800 bits). All numeric arguments (skips, counts, offsets, units) accept multiplication expressions.

--input-skip-units=UNITS

Skip UNITS units (or raw units, if **--input-raw-unit** is set) from the beginning of the input. Supports size suffixes (e.g. 10M, 1G). If both skip-bits and skip-units are defined, both skips are performed. **--input-skip-units** and **--input-skip-bits** automatically seek directly over skipped bytes in O(1) time on regular files rather than reading and discarding data from disk (see **--no-seek**).

--input-gap=BITS

Number of bits skipped after reading each unit. When **--input-raw-unit** is set, defines the gap in bits between successive raw units (default: 0). On seekable files or file descriptors, **--input-gap** and raw unit post-gaps automatically seek directly over gap bytes in O(1) time without buffering.

Raw Units & Containers

In structured binary files, active units often reside inside larger repeating containers, words, or frames (such as 8-bit bytes, 16-bit half-words, or 32-bit words):

--input-raw-unit=BITS

Defines the repeating container or frame size in bits. Within each raw unit, an active unit of **--input-unit** bits is extracted starting at **--input-offset** bits. The remainder of the raw unit (post-gap =

raw-unit - (offset + unit)) is automatically skipped without requiring manual calculation. Extra padding between successive raw units can be specified with `--input-gap`.

`--input-offset=BITS`

Bit offset of the active unit within each raw unit (default: 0). Requires `--input-raw-unit` to be specified.

`--input-assert-aligned`

Gives error message if the input is not full units. Note that with `--count` you can cleanly stop reading non-aligned input.

`--drop-partial-eof, --drop-trailing-bits, --no-pad-eof`

Discard incomplete trailing bits at EOF instead of zero-padding them into an extra synthetic unit.

`--no-seek, --do-not-seek`

Globally disable seeking on all inputs (both primary and merge streams), forcing streaming sequential reads. By default, **bdd** automatically attempts O(1) filesystem seek whenever the input stream is a seekable regular file or file descriptor, seamlessly falling back to sequential stream reads if the input is a pipe, socket, or FIFO.

`--input-no-seek, --no-input-seek`

Disable seeking specifically on the primary input stream, forcing sequential streaming reads.

`--input-use-seek, --input-seek`

Explicitly enable seeking on primary input (this is already the default whenever seeking is possible). Maintained for backwards compatibility.

`--reverse-input-bytes, --input-reverse-bytes`

Every time when a byte is read from input, its bits are reversed (0x01 becomes 0x80). See **--input-little-endian** for details.

`--reverse-input-units, --input-reverse-unit`

Every time when a unit is read, its bits are reversed across the unit. See **--input-little-endian** for details.

`--input-little-endian`

By default, the input is assumed to be in network byte order or big-endian byte order. However, the natural byte order for most modern processors is little-endian. A 16-bit big-endian integer 1 in a byte stream is 0x00 0x01, whereas in little-endian it is 0x01 0x00. In some architectures, even if integers are little-endian, floating-point numbers or strings may follow big-endian order.

The unit is translated as an integer assuming that the least significant bit is first. To implement this as generically as possible, this option is equivalent to combining **--reverse-input-bytes** and **--reverse-input-units**. If we read the 16-bit integer 1 from a little-endian stream, its byte representation is 0x01 0x00. With **--reverse-input-bytes** it becomes 0x80 0x00, and with **--reverse-input-units** it turns into 0x00 0x01. By these two steps, the operation maintains a meaningful, consistent interpretation even for non-byte-aligned units.

After **--input-little-endian**, fields in **--input-pattern** are normally read using uppercase big-endian pattern codes.

Synthetic Stream Sources (Bypass Stream Input)

Instead of reading from a file or pipe, **bdd** can generate synthetic bitstreams directly. The **--count** option specifies how many units the synthetic source produces. If **--count** is omitted (or set to 0), synthetic streams run indefinitely until standard output pipe breaks or the process is terminated.

Values of synthetic inputs are truncated to the input unit size. For example, if you use **--input-counter** with input unit size 8 and output unit size 9, the topmost bit of the output is always zero.

When a synthetic input is used, standard input is not opened for the primary stream, allowing standard input to be used with **--merge-file=-**.

Synthetic inputs do not support reversing bits, initial bit skips, or gap operations. **--input-random**, **--input-counter**, and **--input-integers** support **--skip** to start from an initial offset or value.

--input-zeros

Generate an endless stream of zero bits.

--input-ones

Generate an endless stream of one bits.

--input-random

Generate random bits from `/dev/urandom` (cryptographically strong OS randomness).

--input-counter

Generate sequential counter numbers (0, 1, 2, ...), incrementing by 1 per unit. Counter values are masked to the input unit size. If **--skip=N** is specified, the counter starts at *N*.

--input-integers

Instead of a bit stream, input is text, 10-base unsigned integers separated by newlines. Whitespace around numbers is ignored and empty lines are skipped. Cannot be combined with bit stream unit selection, but can be used with **--skip** and **--count**.

Unpack a unit to a tuple

The input unit is split into fields according the **--input-pattern**. Fields are possibly interpreted as typed values (signed integers, floats or strings).

If there is no **--output-pattern**, the first field of the tuple is output as the sole unit. It must be a positive integer to be interpreted as a unit. If there is no **--input-pattern**, **--output-pattern** can still be used but there is only one field available and it is the input unit as whole.

--input-pattern=PAT

Interpret every unit as fields as described in the PAT. PAT consists of patterns that match some bits in the input unit. A pattern has first the number of the bits matched and then a character that gives those bits the interpretation. For example,

--input-pattern=1U7U

divides a 8-bit unit into two integers, the first bit forms one field and seven next bits form another field. U means that the fields are interpreted as unsigned integers.

x - - - skip these bits without output

U - i - field is unsigned integer (msb first)
 u - i - field is unsigned integer (lsb first)
 M - ii - reads bits as 2's complement signed integer,
 but stores sign and magnitude separately as
 two positive integer fields
 m - ii - same as M, lsb first

M and m generate two fields from one number. The sign is output first as one integer and the magnitude second as another integer. The sign is zero for positive numbers and one for negative numbers. Note that magnitude alone needs as many bits as the original number (look --cut-maxint).

The following type characters are only for rare, advanced use. The main use is that they can give a human interpretation of the input stream using --output-tuples.

If the need is only rearrange bits, you should use only U pattern. The following type characters give some functionality that is sometimes necessary and it makes sense to output them to the bit stream:

S - i - field is 2's complement signed integer (msb)
 s - i - field is 2's complement signed integer (lsb)
 F - d - IEEE-754 32-bit floating point number
 f - d - same, lsb first
 D - d - IEEE-754 64-bit floating point number
 d - d - same, lsb first
 16H - d - IEEE-754 16-bit half precision float (FP16, msb)
 16h - d - IEEE-754 16-bit half precision float (FP16, lsb)
 16Y - d - Google Brain 16-bit bfloat16 (BF16, msb)
 16y - d - Google Brain 16-bit bfloat16 (BF16, lsb)
 8E - d - OCP FP8 E4M3FN (8-bit AI float, msb)
 8e - d - OCP FP8 E4M3FN (8-bit AI float, lsb)
 8Q - d - OCP FP8 E5M2 (8-bit AI float, msb)
 8q - d - OCP FP8 E5M2 (8-bit AI float, lsb)
 6E - d - OCP FP6 E3M2 (6-bit AI float, msb)
 6e - d - OCP FP6 E3M2 (6-bit AI float, lsb)
 4E - d - NVIDIA Blackwell / OCP FP4 E2M1 (4-bit AI float, msb)
 4e - d - NVIDIA Blackwell / OCP FP4 E2M1 (4-bit AI float, lsb)
 c - s - field is unicode UTF-8 character string
 C - s - same, lsb first
 k - i - auto-incrementing unit counter or unsigned integer (lsb)
 K - i - auto-incrementing unit counter or unsigned integer (msb)

Bit widths in pattern specifications are optional. Omitting the bit count applies natural defaults: *B* defaults to 8 bits (identical to 8U or 8B in big-endian bit order; conversely 8u reverses bit order within the byte), *F/f* to 32 bits, *D/d* to 64 bits, *H/h/Y/y* to 16 bits, *E/e/Q/q/C/c/K/k* to 8 bits, *m* to 4 bits, and single-bit types *U/u/S/s/x/X/z/o/r* default to 1 bit.

Patterns also support repetition multipliers in the format *COUNT*SPEC* or *COUNT*(SPEC1 SPEC2 ...)*. A multiplier creates *COUNT* separate fields in the tuple, not one combined integer. For example, 8*3U expands to eight separate 3-bit fields 3U3U3U3U3U3U3U3U (producing an 8-field tuple, total 24 bits), whereas 24U produces a single 24-bit integer field. Likewise, 4*8B or 4*B expands to four 8-bit unsigned bytes, and 2*(4U4u) expands to 4U4u4U4u. Patterns also support named fields in the format *name:SPEC* or *name:COUNT*SPEC* (e.g. sync:11u, version:2u or w:4*6E which expands to w_0:6E, w_1:6E, w_2:6E, w_3:6E). Field names automatically label JSON object keys with --json-object and generate CSV header lines.

--preset=NAME

Select a predefined binary protocol or AI float format preset. Automatically configures `--input-pattern`, `--input-unit`, and field names. Supported presets include **mp3-header**, **mpeg-ts**, **wav-header**, **jpeg-sof0**, **h264-nal**, **nvfp4**, **fp6-e3m2**, **fp8-e4m3**, **fp8-e5m2**, **bf16**, **fp16**, **ipv4-header**, **udp-header**, **tcp-header**, and **riscv-r-type**.

--list-presets

Print a formatted table of all available built-in presets with pattern definitions, unit widths, and descriptions, then exit.

S and s should be used with caution. If negative value is used in making a unit, it will cause an error. The only way to use negative values correctly is `--output-pattern=nS`, `--abs` or `--output-tuples`.

Floating-point numbers can be read and written using any of the supported float patterns: 64D/64d (FP64), 32F/32f (FP32), 16H/16h (IEEE FP16), 16Y/16y (Google Brain BF16), 8E/8e (OCP FP8 E4M3FN), 8Q/8q (OCP FP8 E5M2), 6E/6e (OCP FP6 E3M2), and 4E/4e (NVIDIA Blackwell / OCP FP4 E2M1). Internally, all input floats are decoded into IEEE 754 64-bit doubles (*f64*) in the tuple field pipeline. When written out through an output pattern, *bdd* encodes the internal *f64* value directly into the requested output float format using standard round-to-nearest-even rules (or round-to-nearest). This provides native, loss-free upcasting and bit-exact downcasting/quantization between arbitrary floating-point representations (e.g. `--input-pattern=32F --output-pattern=16H` or `--input-pattern=16H --output-pattern=4E`). The `--round` manipulator can additionally be used to apply explicit rounding modes (such as **floor**, **ceil**, **trunc**, or **round_ties_even**) prior to packing.

To verify or guarantee that floating-point numbers copy bit-for-bit verbatim as-is without any possible intermediate rounding to/from *f64* (preserving exact NaN payloads and subnormals), handle them as unsigned integers of the equivalent bit width: **32U** for 32-bit floats, **64U** for 64-bit doubles, **16U** for FP16/BF16, or **8U** for FP8. Unsigned integer fields copy raw bit patterns verbatim without float interpretation.

Most likely, you want to use upper case variants of the patterns (bitwise big-endian representation). If you want to interpret bitwise little-endian numbers, you likely want to use `--input-little-endian` and still use uppercase patterns. Lowercase patterns are for the (rare) case that the value in bit stream is exactly in the bitwise (not bitwise) opposite order, least significant bit first.

Rule to remember: uppercase letters are big-endian, lowercase letters are little-endian.

Floating point number length must match the bit length. D must be always written as pattern 64D.

Also, the string must be right amount of bytes. Some byte strings can not be output as a valid UTF-8 string.

Bypass unit and field selection**--input-tuples**

Instead of a bit stream, input is a text file containing tuples. Tuple is a line of comma-separated values like:

```
123, -55, 678.5, "abba"
```

Tuples are separated with newline. Whitespace around the comma is allowed. If `--input-tuples` is used, unit or field selection arguments have no effect but `--count`, `--skip` and tuple manipulation options can be used.

Allowed types and their internal representations are:

Integers (signed bignums)
 Floats (translated as IEEE 754 64-bit floating point numbers)
 Strings (UTF-8, enclosed in quotes like "123" to prevent numeric coercion)

Manipulate tuple

Fields are represented with field numbers (starting from 0, negative indices count from the end with -1 being the last field). The manipulation pipeline is ordered and stackable: operations execute in the exact order they are specified on the command line. Multiple operations (such as arithmetic, bit shifts, logical masks, filtering, and reordering) can be chained together.

`--rearrange=FIELDLIST`

Selects fields from input tuple for output tuple. Comma-separated list of field numbers starting from 0 (field 0 is the first field in the tuple). Negative field number refers to the field counting from the end of the tuple (-1 is the last field). The same field number can exist several times duplicating the input field. Not all input fields need to be in the output tuple.

`--add=FIELD,VALUE`

Adds VALUE to the numerical value of FIELD.

`--sub=FIELD,VALUE`

Subtracts VALUE from the numerical value of FIELD.

`--mul=FIELD,VALUE`

Multiplies FIELD by VALUE.

`--div=FIELD,VALUE`

Performs integer or floating-point division of FIELD by VALUE.

`--mod=FIELD,VALUE`

Computes modulo remainder of FIELD by VALUE.

`--shift-left=FIELD,BITS`

Shifts FIELD left by BITS bits (multiplies by 2^{BITS}).

`--shift-right=FIELD,BITS`

Shifts FIELD right by BITS bits (integer division by 2^{BITS}).

`--and=FIELD,MASK`

Performs bitwise AND of FIELD with MASK (decimal, 0x hex, or 0b binary).

`--or=FIELD,MASK`

Performs bitwise OR of FIELD with MASK (decimal, 0x hex, or 0b binary).

`--xor=FIELD,BITS`

Inverts the rightmost BITS bits of FIELD, or applies a direct bitwise XOR mask if specified in hex or binary format. For example, the sign of a number read using pattern "M" can be changed using `--xor=0,1`.

`--not=FIELD`

Inverts all bits of FIELD.

`--abs=FIELD`

Removes sign from the value of the field (calculates absolute value). Legacy syntax

`--abs=FIELD,0` is also accepted.

`--sign=FIELD`

Extracts the sign of the field: 1 if the value was negative and 0 if non-negative. Legacy syntax

`--sign=FIELD,0` is also accepted.

`--filter=FIELD,OP,VALUE`

Filters tuples based on a predicate condition where OP is one of `==`, `!=`, `<`, `<=`, `>`, or `>=`. Tuples failing the predicate are discarded from the output stream.

`--round=FIELD,LIMIT[,MODE] | --round=FIELD,MODE`

Rounds floating-point fields or bounds/clamps integer fields within `[-LIMIT, LIMIT]` (or `[0, LIMIT]` if unsigned). Visible alias: `--cut-maxint`. When LIMIT is omitted (e.g. `--round=0, floor` or `--round=0, round_ties_even`), the rounding mode is applied directly without magnitude clamping. An optional MODE parameter may be appended using comma or colon delimiter (e.g. `--round=0, 127, wrap` or `--round=0, 127:round_ties_even`). Supported modes:

- **saturate** or **clamp** (default): Clamps out-of-range values to MAXINT or -MAXINT.
- **wrap** or **wrapping**: Wraps values around using modular arithmetic (modulo MAXINT+1 for unsigned, or across `[-MAXINT, MAXINT]` for signed).
- **zero** or **reset**: Replaces out-of-range values with zero.
- **drop** or **filter**: Discards the entire tuple from the output stream if value exceeds bounds.
- **trunc**: Truncates magnitude toward zero.
- **floor**: Rounds toward negative infinity.
- **ceil**: Rounds toward positive infinity.
- **round**: Rounds to nearest neighbor, ties away from zero.
- **round_ties_even**: Rounds to nearest neighbor, ties to even digit (Rust and IEEE 754 default banker's rounding).

Field number can be negative when the position is counted from the end of the tuple.

`--remove-right=FIELD,BITS`

If we want to output other bits from the field than the rightmost ones, we can remove BITS rightmost bits from the field.

Skip a part of tuples

`--skip=N`

N first units are skipped.

`--count=N`

Only N units are processed. For example, if `--skip=1` and `--count=1`, only the second unit is processed.

Pack a tuple to a unit**--output-pattern='PAT'**

Packs output tuple to output bit unit. Fields must be in order, use --rearrange to arrange fields to the order of output pattern. Types are the same as the ones with the input pattern with the following exceptions:

- z - fill with zero
- o - fill with one
- r - fill with random (from /dev/urandom)
- x - consume and discard one field from the tuple without emitting bits
- k - emit auto-incrementing sequence counter (lsb first)
- K - emit auto-incrementing sequence counter (msb first)

Also in the output-pattern, lowercase letters turn the bit order reversed. Look --input-pattern for more information.

z, o, r, k and K do not consume fields from the tuple. x and X consume one field from the tuple and discard it without emitting bits. m and M consume two fields from the tuple and convert them to one 2's complement bit unit.

If signed value is too big to a signed field, the magnitude is first truncated to the right size. 4-bit signed output of -17 is '1111', because 17 is first truncated to 4 bits giving '0001' or 1 and the bit representation of -1 is '1111'.

If tuples are used (--input-pattern or --input-tuples), but there is no --output-pattern option, the first field in the tuple is used directly as the output unit. The first field of the tuple must be an unsigned integer if it is used as an output unit.

Collect output data stream**--output-unit=BITS**

Packs the output unit to BITS bits. If --output-pattern is defined, output unit size comes from the total size of the output pattern and --output-unit must not be defined. If there is neither --output-unit nor --output-pattern, the default output unit size is 8 bits.

The last output byte is filled with zeros, if needed to align to an 8-bit byte boundary.

Warning on Sub-Byte Streams: When packing smaller units into bytes (for example, 1-bit or 3-bit output units), the remaining bits in the final flushed byte are zero-padded. For instance, if the stream terminates after writing three 1-bit units, **bdd** flushes a full 8-bit byte containing those three bits followed by five zero bits. When that file is subsequently read as bits (or with **--input-unit=1**), the reader will observe eight bits (the three original units plus five extra zero bits). *The count of what you write is not necessarily the count of what you read.* If exact unit preservation is important, you have to count units yourself and supply **--count=N** when reading back the stream, or store the length in external metadata.

--output-raw-unit=BITS

Size of repeating raw unit / container in bits on output. When specified, the active output unit is framed within the container at --output-offset, and any remaining bits (post-gap = raw_unit - offset - unit) are filled with zeros.

--output-offset=BITS

Bit offset of the active unit within the output raw container (default: 0). Requires --output-raw-unit to be specified.

--output-gap=BITS

Bit gap in zeros emitted between successive output raw unit containers (default: 0).

--output-skip-bits=BITS, --output-skip, --output-prefix

Initial prefix of zero bits emitted on output before the first unit is written (default: 0).

--output-little-endian

Unit is written to the output stream least significant byte first. The default is network byte order or big-endian byte order where the most significant byte is written first. **--output-little-endian** is a combination of **--reverse-output-units** and **--reverse-output-bytes**. See **--input-little-endian** for details.

--reverse-output-units, --output-reverse-unit

Reverses bits in the output unit. See **--output-little-endian** for details.

--reverse-output-bytes, --output-reverse-bytes

Reverses bits of every written byte. See **--output-little-endian** for details.

Stream Merging & Interleaving

Units of the merge file are written to the output after every primary output unit. Processing terminates when the primary output stream finishes. A final merge unit is written after the last primary output unit, maintaining an equal count of primary and merge units. It is an error if the merge file does not contain enough units, but it may contain extra units, which are ignored.

Merge unit is defined like the normal input unit. Merge unit is written to the output with the same unit size as it has in input. Merge unit size can be different than the normal output unit. Output of the merge unit is done with the same **--reverse-bytes** and **--reverse-unit** options as the normal output.

--merge-file=FILENAME

Opens the merge file. Can be specified multiple times for N-way round-robin interleaving with the primary stream. If FILENAME is "-", mergefile is standard input. If mergefile is standard input, input file must not be the same.

--merge-files=FILENAMES

Comma- or space-separated list of merge files to interleave round-robin with the primary input stream.

--merge-unit=BITS

Breaks merge input to units of BITS.

--merge-skip-bits=BITS

Skip BITS bits from the beginning of the merge input

--merge-skip-units=UNITS

Skip UNITS units from the beginning of the merge input.

--merge-copy-first=BITS

Copy BITS bits as-is from the beginning of the merge file before writing the first unit. If both skip (bits and/or units) and **--merge-copy-first** are defined, skip is done first.

--merge-gap=BITS

When --merge-raw-unit is set, defines the gap in bits between successive merge raw units (default: 0). When --merge-raw-unit is omitted, specifies the bits skipped after writing each unit.

--merge-raw-unit=BITS

Defines the repeating container or frame size in bits for the merge stream. Within each merge raw unit, an active unit of --merge-unit bits is extracted starting at --merge-offset bits. The remainder of the merge raw unit is automatically skipped.

--merge-offset=BITS

Bit offset of the active unit within each merge raw unit (default: 0). Requires --merge-raw-unit to be specified.

--merge-assert-aligned

Warn if the whole merge input was not used or the merge input did not end to the byte border.

--merge-drop-partial-eof, --merge-drop-trailing-bits, --merge-no-pad-eof

Discard incomplete trailing bits at EOF in merge stream instead of zero-padding.

--merge-no-seek, --no-merge-seek

Disable seeking specifically on the merge input stream, forcing sequential streaming reads. By default, seeking is automatically attempted whenever the merge source is a seekable regular file.

--merge-use-seek, --merge-seek

Explicitly enable seeking on merge input (this is already the default whenever seeking is possible). Maintained for backwards compatibility.

--merge-little-endian

Merge file is read with both --merge-reverse-bytes and --merge-reverse-unit on.

--merge-reverse-bytes

Every byte read from merge file is reversed.

--merge-reverse-unit

Every unit read from merge file is reversed.

Human-Readable and Text Output Sinks**--output-tuples**

Replace bit output with human-readable output of tuples. This output is readable with --input-tuples

--output-integers

Replace bit output with human-readable output of units as decimal integers (one per line). This output is readable with --input-integers

If merge file is used, the main unit and the merge unit are printed separately as two integers (possibly with different unit sizes).

The integer is truncated to the unit size, and bit reversals do not affect the decimal integer output.

--output-hex

Replace bit output with human-readable output of units as hexadecimal numbers.

Units are formatted as they would be written to the bit stream, except that `--output-little-endian`, `--output-reverse-bytes`, and `--output-reverse-unit` do not affect hex output.

Output unit size affects the number as it affects bit output. If output unit size is 3 bits, the number 8 is printed as hex 0. The hex number has as many digits as needed to represent the full unit, but topmost zero bits are not necessarily a part of the output unit. Units from the merge file are written with their own output unit so that they may have a different number of digits.

Output is divided into lines formatted so that each line contains 1, 2, 4, 8, 16, or 32 units separated by spaces. The line length is kept between 40 and 80 characters.

--output-bits

Replace bit output with human-readable output of units as ASCII bit strings ('0' and '1'). See **--output-hex** for formatting details.

--output-json

Output stream of structured JSON arrays or objects (NDJSON), one record per line. When named patterns, presets, or `--json-fields` are used, emits keyed JSON dictionaries.

--json-object

Force JSON output to be emitted as keyed JSON objects instead of positional arrays. Field keys default to pattern names (e.g. `sync:11u`), custom `--json-fields`, or auto-generated `field_0`, `field_1`, ... keys.

--json-fields=NAME1,NAME2,...

Explicit comma-separated list of field names used for JSON object keys and CSV column headers.

--output-csv

Output stream as comma-separated values (CSV).

--csv-header=COL1,COL2,...

Specify custom column names for CSV output header row.

--output-visual

ANSI color-coded visual stream inspector highlighting bit boundaries and field values.

Stream demuxing and routing**--demux=FIELD:PATH**

Extract individual tuple field by index and stream it directly to a dedicated file or FIFO. Can be repeated.

--demux-files=PATH1,PATH2,...

Demux tuple fields in order to a comma-separated list of target files.

--input-repeat=COUNT

Loop through the input stream COUNT times (supports seeking or memory buffering).

Analysis, Inspection and Model Context Protocol (MCP)

--explain-pattern[=PATTERN]

Analyze and display the structural breakdown of PATTERN (or the pattern set via --preset or --input-pattern), reporting total bits, bytes, byte alignment, and per-field bit ranges, byte offsets, and data types. If --output-json is specified, produces structured JSON output suitable for AI tooling and IDE extensions.

--probe[=FILENAME]

Deeply inspect the input file (or standard input if omitted). Computes Shannon entropy, byte class frequencies (nulls, printable ASCII, high bytes), periodic stride autocorrelation across offsets 1..512 bytes (detecting periodic formats like 188-byte MPEG-TS packets, 24-bit audio, or fixed-width telemetry frames), and extracts prominent ASCII string runs. If --output-json is specified, outputs full structured JSON metrics.

--mcp Run native JSON-RPC 2.0 Model Context Protocol (MCP) server over standard input and standard output. Exposes four tools for AI agent pair programming, binary reverse engineering, and pipeline automation: **bdd_slice** (extract, transcode, and slice bitstreams into text or JSON), **bdd_probe** (probing binary blobs for entropy and periodicity), **bdd_explain_pattern** (inspecting schema bit layouts), and **bdd_list_presets** (cataloging built-in protocols).

--llms, --ai-guide, --ai

Print the embedded concise AI model and agent guide (llms.txt) to standard output. Contains bitstream syntax rules, pattern grammar, and pipeline recipes for language models.

--serve[=PORT], --web, --gui

Launch the interactive Web UI application server (default port 7788). Allows exploring bit layouts, running slicing pipelines, visualizing hex dumps, and testing presets directly in any modern web browser.

-q, --quiet

Quiet mode. Suppress non-fatal diagnostic warnings (such as missing tuple fields or stream conversion notices) and the EOF warning summary. Diagnostic warnings are automatically deduplicated during stream processing, and --quiet silences them entirely for maximum pipeline throughput.

UNIFIED STREAM I/O MAPPING SYNTAX

Instead of specifying separate flags for skips, containers, offsets, and gaps, **bdd** accepts a unified, expressive stream I/O mapping expression as an optional positional argument:

bdd "<input_stream> -> <output_stream>" [*input_pattern*] [*output_pattern*] [*options*]

Each stream specification follows the syntax:

[skip :] unit_or_container [+ gap]

- **Initial Skip / Prefix (:)**: Delimited by :. Skips *N* bits on input, or emits *N* zero bits as an initial stream prefix on output. Supports unit/byte multipliers (e.g. 123, 5B, 4k).
- **Periodic Gap (+)**: Appended with +. Bits skipped between input containers, or zero bits emitted between output containers.
- **Form A (Container Slicing)**: *raw_size*[*offset* : *unit*] — container size outside, offset and active unit inside. Post-gap is calculated automatically as *raw_size* - *offset* - *unit*.

- **Form B (Physical Layout Box):** `[pre : unit : post]` — explicit pre-gap, active payload, and post-gap inside the box. Container size is the sum `pre + unit + post`.
- **Bare Unit:** Plain number or size (e.g. 8, 3, 188B).

All flags (`--input-raw-unit`, `--output-raw-unit`, etc.) remain fully functional and can override or complement positional arguments.

EXAMPLES

`bdd 8 -> 3 < foo.8bit > foo.3bit`

Packs 8-bit bytes into a dense stream of 3-bit units (top 5 bits ignored).

`bdd 123:8[2:4]+8 -> 5B:8[2:4] < in.bin > out.bin`

Skips 123 bits, slices 4-bit payload from 8-bit containers (offset 2, post-gap 2) with 8-bit gaps between containers, then emits 5 zero bytes on output and frames each unit inside an 8-bit container (offset 2, post-gap 2).

`bdd 188B[11:13] -> 13 < broadcast.ts > pids.bin`

Extracts 13-bit MPEG-TS PID fields directly from 188-byte packet streams.

`bdd 4U4U 4U4U --rearrange=1,0 < in.bin > out.bin`

Unpack each binary byte into two 4-bit nibbles, swap fields, and repack into bytes.

`bdd 4U4U --input-tuples --output-hex < pairs.txt`

Read comma-separated text pairs ("1,2"), pack each pair into an 8-bit byte.

`bdd < foo > bar`

Just copies file foo to bar, one byte at time.

`bdd --input-unit=3`

Extracts 3-bit stream values from standard input and expands values to bytes in standard output. If the file is not aligned to 3 bits, the missing least significant bits for the last unit are zeros.

`bdd --input-unit=1 --output-pattern=1U1z`

Output stream is input stream with zero added after every bit.
Four two-bit units are packed to one byte.

`bdd --input-pattern=8U8U --rearrange=1,0 --output-pattern=8U8U`

Swap two consecutive bytes.

`echo -en '1,2,30,7,7' | bdd --input-tuples --output-pattern='2U3U3U' | od -t o1`

Build octal numbers from fields (prints 123 377)

`bdd --input-counter --count=16 --output-hex`

Prints:

00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f

bdd --input-counter --count=16 --input-unit=12 --output-unit=12 --merge-file=/etc/passwd --output-hex

Prints first 16 bytes of /etc/passwd file as hex preceded by a three-digit hex address of the byte:

```
000 72 001 6f 002 6f 003 74 004 3a 005 78 006 3a 007 30
008 3a 009 30 00a 3a 00b 72 00c 6f 00d 6f 00e 74 00f 3a
```

FILES

/usr/share/doc/bdd/lms.txt

Concise AI model and agent guide containing bitstream syntax rules, pattern grammar, and pipeline recipes.

/usr/share/doc/bdd/README.md

Comprehensive project documentation and architecture guide.

/usr/include/bdd.h

C-ABI shared library header file.

SEE ALSO

dd(1)

BUGS

None known in the current release. Please report issues at <https://github.com/e-t-u/bdd>.

AUTHOR

Esa Turtiainen (esa<at>turtiainen<dot>eu)